

Computação Gráfica



Professor: EDILSON LIMA

ATIVIDADE PRÁTICA – Transformações Geométrica

Atividade Prática: Design de Interface para um Painel Urbano Interativo

Ferramenta sugerida: Google Colab

1. Escala de um objeto 2D (pontos do slide)

Situação-problema:

Um designer quer aumentar um ícone 2D mantendo suas proporções. Os pontos do ícone são: (2,1), (3,1), (2,2), (3,2). Ele deseja aplicar uma escala de ($s_x = 3$) e ($s_y = 4$).

“A transformação de escala de um ponto (x_1, y_1), por uma escala (s_x, s_y), se dá pelas equações $x_2 = s_x x_1$, $y_2 = s_y y_1$.”

Tarefa:

- Implementar a matriz de escala homogênea 2D:

$$S = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Aplicar em todos os pontos e plotar antes/depois.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Pontos originais (homogêneos)
```

```
pts = np.array([
    [2, 3, 2, 3],
    [1, 1, 2, 2],
    [1, 1, 1, 1]
], dtype=float)
```

```
sx, sy = 3, 4
```

```
S = np.array([
    [sx, 0, 0],
```

```
[0, sy, 0],  
[0, 0, 1]  
, dtype=float)
```

```
pts_scaled = S @ pts
```

```
plt.figure()  
plt.scatter(pts[0], pts[1], c='blue', label='Original')  
plt.scatter(pts_scaled[0], pts_scaled[1], c='red', label='Escalado')  
plt.axhline(0, color='black'); plt.axvline(0, color='black')  
plt.legend(); plt.gca().set_aspect('equal', 'box')  
plt.show()
```

2. Translação de um objeto 2D

Situação-problema:

Um personagem 2D precisa “andar” na tela. O ponto inicial é $((x_1, y_1) = (3, 2))$ e ele deve ser transladado por $((t_x, t_y) = (3, 6))$, indo para $((6, 8))$, como no slide.

“A translação de um ponto (x_1, y_1) por (t_x, t_y) se dá pelas equações $x_2 = x_1 + t_x$, $y_2 = y_1 + t_y$.”

Matriz de translação homogênea 2D:

$$T = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

```
import numpy as np
```

```
p1 = np.array([[3],[2],[1]], dtype=float)
tx, ty = 3, 6
```

```
T = np.array([
    [1, 0, tx],
    [0, 1, ty],
    [0, 0, 1 ]
], dtype=float)
```

```
p2 = T @ p1
print("Ponto original:", p1.ravel()[:2])
print("Ponto transladado:", p2.ravel()[:2])
```

3. Rotação 2D em torno da origem e em torno de um ponto

Situação-problema:

Um objeto 2D (por exemplo, um quadrado) deve girar em torno da origem e depois em torno de um ponto ($c = (x_c, y_c)$), como no slide de composição.

“Logo, a matriz de rotação 2D em torno da origem é

$$R = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

(

a) Rotação em torno da origem

```
import numpy as np
import matplotlib.pyplot as plt
```

```
def R2d(theta):
    c, s = np.cos(theta), np.sin(theta)
    return np.array([
        [c, -s, 0],
        [s, c, 0],
        [0, 0, 1]
    ], dtype=float)
```

```
# Quadrado
pts = np.array([
    [1, 2, 2, 1],
    [1, 1, 2, 2],
    [1, 1, 1, 1]
], dtype=float)
```

```
theta = np.deg2rad(45)
R = R2d(theta)
pts_rot = R @ pts
```

```
plt.figure()
plt.scatter(pts[0], pts[1], c='blue', label='Original')
plt.scatter(pts_rot[0], pts_rot[1], c='red', label='Rotacionado')
plt.axhline(0, color='black'); plt.axvline(0, color='black')
plt.legend(); plt.gca().set_aspect('equal', 'box')
plt.show()
```

b) Rotação em torno de um ponto ($c = (x_c, y_c)$)

“Fazer a translação de $-c$, depois a rotação, depois a translação de c .”

```
[ R_c = T_c , R , T_{-c} ]
```

```
def T2d(tx, ty):
    return np.array([
        [1, 0, tx],
        [0, 1, ty],
```

```
[0, 0, 1 ]  
, dtype=float)
```

```
cx, cy = 1.0, 2.0  
Rc = T2d(cx, cy) @ R2d(theta) @ T2d(-cx, -cy)  
pts_rot_c = Rc @ pts
```

```
plt.figure()  
plt.scatter(pts[0], pts[1], c='blue', label='Original')  
plt.scatter(pts_rot_c[0], pts_rot_c[1], c='green', label='Rot. em torno de c')  
plt.scatter([cx],[cy], c='red', label='Centro c')  
plt.axhline(0, color='black'); plt.axvline(0, color='black')  
plt.legend(); plt.gca().set_aspect('equal', 'box')  
plt.show()
```

4. Movimento das orelhas e do olho do macaco 3D (Suzanne)



Situação-problema:

Você tem o modelo 3D do macaco (Suzanne) e quer animar **apenas as orelhas** (balançando) e **um olho** (olhando para cima/baixo), usando **transformações geométricas 3D** (rotação + translação).

“A matriz de rotação é igual à matriz 2D, apenas acrescida da dimensão Z.”

Ideia didática (sem depender do Blender dentro do Colab):

- Representar as orelhas e o olho como **conjuntos de pontos 3D** (por exemplo, pequenos cubos ou esferas).
- Aplicar:
 - Rotação em torno de um eixo que passa pela base da orelha.
 - Rotação do olho em torno do seu centro.

Exemplo simplificado: orelha como um pequeno conjunto de pontos 3D

```
import numpy as np
import matplotlib.pyplot as plt
```

```
def Rz(theta):
```

```
    c, s = np.cos(theta), np.sin(theta)
```

```
    return np.array([
```

```
        [c, -s, 0, 0],
```

```
        [s, c, 0, 0],
```

```
        [0, 0, 1, 0],
```

```
        [0, 0, 0, 1]
```

```
    ], dtype=float)
```

```
def T3d(tx, ty, tz):
```

```
    return np.array([
```

```
        [1, 0, 0, tx],
```

```
        [0, 1, 0, ty],
```

```
        [0, 0, 1, tz],
```

```
        [0, 0, 0, 1]
```

```
    ], dtype=float)
```

```

# "Orelha" como pontos em torno de (2,2,0)
ear = np.array([
    [2, 2.2, 1.8, 2],
    [2, 2.1, 2.1, 1.8],
    [0, 0, 0, 0],
    [1, 1, 1, 1]
], dtype=float)

# Centro da orelha
cx, cy, cz = 2.0, 2.0, 0.0

def rotate_ear(ear, angle):
    M = T3d(cx, cy, cz) @ Rz(angle) @ T3d(-cx, -cy, -cz)
    return M @ ear

angles = np.linspace(-0.5, 0.5, 5)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

for a in angles:
    ear_rot = rotate_ear(ear, a)
    ax.scatter(ear_rot[0], ear_rot[1], ear_rot[2])

ax.set_xlabel('X'); ax.set_ylabel('Y'); ax.set_zlabel('Z')
plt.show()

```

Para o olho, o aluno pode repetir a lógica, mas rotacionando em torno de outro centro e talvez em torno do eixo X ou Y (olhar para cima/baixo).

5. Transformação de perspectiva e projeção

Situação-problema:

Uma câmera está posicionada no eixo Z, a uma distância (D) da origem. Você quer projetar um cubo 3D no plano de projeção (imagem 2D), usando a matriz de perspectiva dos slides.

“Para estas configurações, as matrizes de transformação perspectiva que projeta o ponto p sobre o plano de projeção são...”

Para centro em ((0,0,-D)), a matriz (P) (forma homogênea) é:

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{D} & 0 \end{bmatrix}$$

(de acordo com a forma apresentada no slide, adaptando para coordenadas homogêneas, o importante é que depois fazemos ($x = X'/H$, $y = Y'/H$)).

Tarefa:

- Definir um cubo 3D (8 vértices).
- Aplicar a matriz de perspectiva.
- Fazer a divisão por (H) para obter ((x,y)).
- Plotar o cubo projetado em 2D.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Vértices de um cubo centrado na origem
```

```
cube = np.array([
    [-1, -1, -1, 1],
    [ 1, -1, -1, 1],
    [ 1,  1, -1, 1],
    [-1,  1, -1, 1],
    [-1, -1,  1, 1],
    [ 1, -1,  1, 1],
    [ 1,  1,  1, 1],
    [-1,  1,  1, 1]
], dtype=float).T # 4x8
```

```
D = 5.0
```

```
P = np.array([
    [1, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [0, 0, 1/D, 0]
], dtype=float)
```

```
proj = P @ cube
Xp = proj[0] / proj[3]
Yp = proj[1] / proj[3]
```

```
plt.figure()
plt.scatter(Xp, Yp)
for i in range(8):
    plt.text(Xp[i], Yp[i], str(i))
plt.axhline(0, color='black'); plt.axvline(0, color='black')
plt.gca().set_aspect('equal', 'box')
plt.title('Projeção em perspectiva do cubo')
plt.show()
```